Csrf (Cross-Site Request Forgery) - klasa napada (Clickjacking lab)

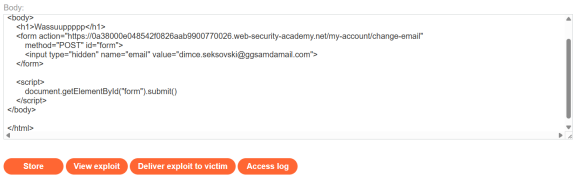
Sergej Mika

21. april 2026.

1. Lab: CSRF vulnerability with no defenses

Moramo da napravimo HTML template, koji ćemo poslati victim-u. Koristićemo njegov cookie i znanje o request-u i svakom njegovom parametru kako bi exploit-ovali promenu email adrese korisnika.

Slika 1: Prvo se login-ujemo u našeg korisnika i promenimo email kako bi u Burp Suite pogledali strukturu request-a i znali kako da napravimo HTML template.

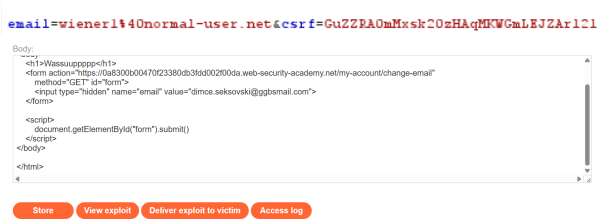


Slika 2: Na osnovu request-a pravimo template koji će se izvršiti odma pri otvaranju. Stavljamo sve parametre bez cookie jer će browser sam poslati cookie. Jedini parametar je email koji postavljamo na nama željeni mail. Nakon toga pritiskamo dugme Deliver exploit to victim, gde sceniramo da je korisnik koga smo želeli da napadnemo kliknuo link.

2. Lab: CSRF where token validation depends on request method

Pošto sada imamo generisanje csrf tokena kako bi sprečili csrf ponovićemo postupak kao i u prošlom zadatku. Uradićemo prvo request na našem korisniku kako bi otkrili

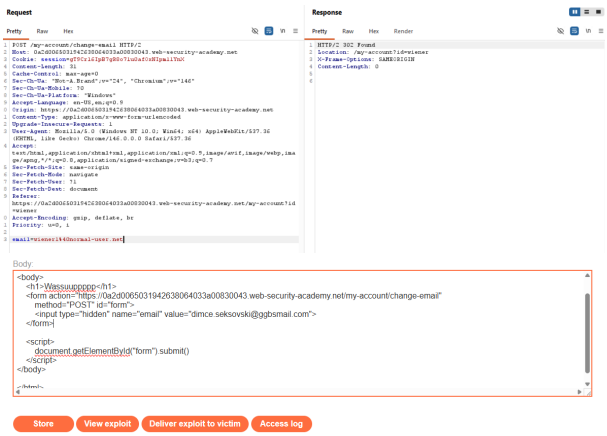
strukuturu request-a i saznajemo da kao parametar sem email imamo i csrf. Pošto je struktura HTML template-a ista kao i u prvom zadatku, ako pokušamo da izvršimo napad sa tim template-om dobijamo error da nismo uneli parametar csrf, ali kada promenimo template sa POST na GET metodu, napad prolazi pošto su developeri napravili propust i nisu stavili proveru csrf tokena pri GET metodi.



Slika 3: Parametar csrf u POST request-u, novi template sa GET metodom

3. Lab: CSRF where token validation depends on token being present

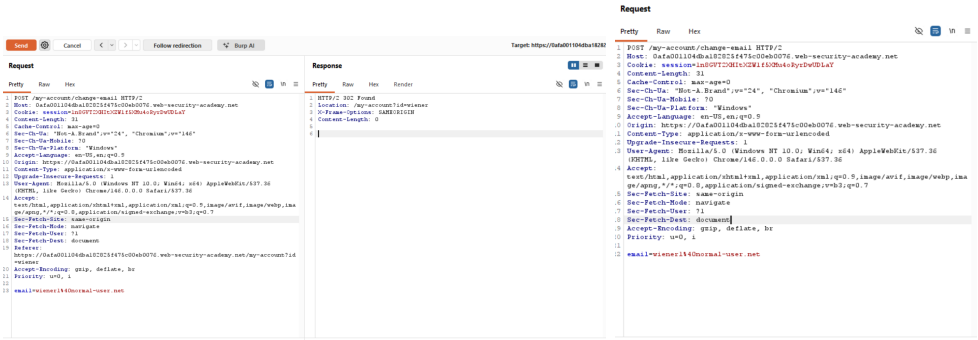
Sada pokušavamo da iskoristimo vulnerability kada se ne navede csrf token isti neće ni biti validiran. Prvo pošaljemo opet request preko našeg poznatog korisnika, dobijamo strukturu request-a gde vidimo da šaljemo email i csrf. Ali kada u Repeater-u pošaljemo request bez csrf tokena, request prolazi što znači da možemo da ponovimo HTML template iz prvog zadatka kako bi izvršili napad.



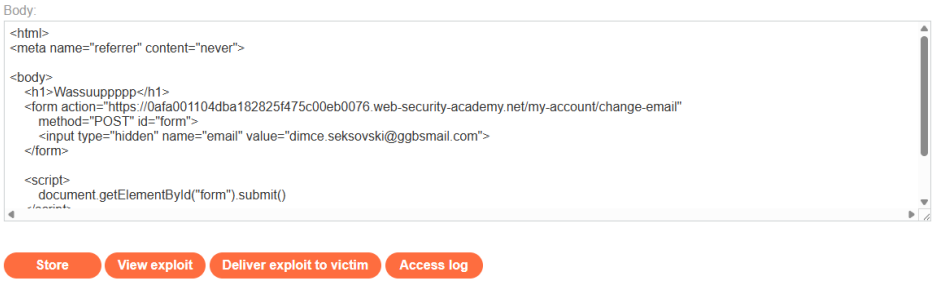
Slika 4: Request koji prolazi bez csrf tokena, HTML template

4. Lab: CSRF where Referer validation depends on header being present

Pošto u nekim situacijama koje su mnogo ređe, developeri koriste referer header field umesto csrf tokena kako bi pokušali da spreče csrf napad. Server proverava Referer header kako bi se uverio da request zaista dolazi sa njegovog sopstvenog domena, a ne sa nekog zlonamernog sajta koji pokušava da izvede CSRF napad. Vulnerability u ovom primeru je da ako ne navedemo Referer header u request-u neće ni biti validiran.



Slika 5: U Repeateru isprobamo request sa i bez Referer header-a i oba prolaze.



Slika 6: U HTML template dodajemo meta tag kako ne bi slali Referer header u request-u pri CSRF napadu.

5. CSRF - klasa napada

CSRF je tip napada koji prisiljava krajnjeg korisnika da izvrši neželjene radnje na veb aplikaciji u kojoj je trenutno autentifikovan. Za razliku od krađe podataka, kod CSRF-a napadač cilja na promenu stanja na serveru, kao što su promena lozinke, prenos sredstava ili slanje poruka, koristeći pritom žrtvin identitet.

5.1 Opis klase napada

Suština CSRF napada leži u načinu na koji veb pregledači rukuju cookies-ima. Kada se korisnik uloguje na sajt, pregledač čuva session cookie. Problem nastaje jer pregledač automatski prilaže taj kolačić uz svaki zahtev koji se šalje tom sajtu, bez obzira na to odakle je zahtev iniciran. Napadač postavlja malicioznu skriptu ili link na treći sajt (npr. forum ili lažni oglas). Kada žrtva poseti taj sajt dok je još ulogovana na ciljanu aplikaciju, skripta u pozadini šalje HTTP zahtev ka toj aplikaciji. Pošto pregledač vidi da cookies-i postoje, on ih šalje uz zahtev, i aplikacija ga izvršava misleći da ga je svesno poslao korisnik.

5.2 Exploit impact

Uticaj CSRF-a može varirati od manje neprijatnosti do potpunog gubitka kontrole nad aplikacijom:

- **Promena kredencijala:** Napadač može promeniti email adresu ili lozinku žrtve, čime trajno preuzima nalog.

- **Finansijske transakcije:** U bankarskim aplikacijama, CSRF se može iskoristiti za neovlašćeni transfer novca.
- **Administrativne akcije:** Ako je žrtva administrator, napadač može kreirati nove korisničke naloge, obrisati podatke ili promeniti globalna podešavanja sistema.

5.3 Ranjivosti u softveru koje dozvoljavaju napad

Da bi CSRF napad bio moguć, moraju biti ispunjena tri ključna uslova:

- **Relevantna akcija:** Postoji funkcionalnost koju napadač želi da zloupotrebi (npr. prebacivanje novca ili promena podataka).
- **Cookie-based session management:** Aplikacija se oslanja isključivo na HTTP kolačiće za identifikaciju korisnika.
- **Predvidljivi parametri zahteva:** Zahtev ne sadrži nikakve tajne parametre koje napadač ne može da pogodi ili generiše (npr. 'POST /update-email?email=dimce@mail.com').

5.4 Kontramere i prevencija

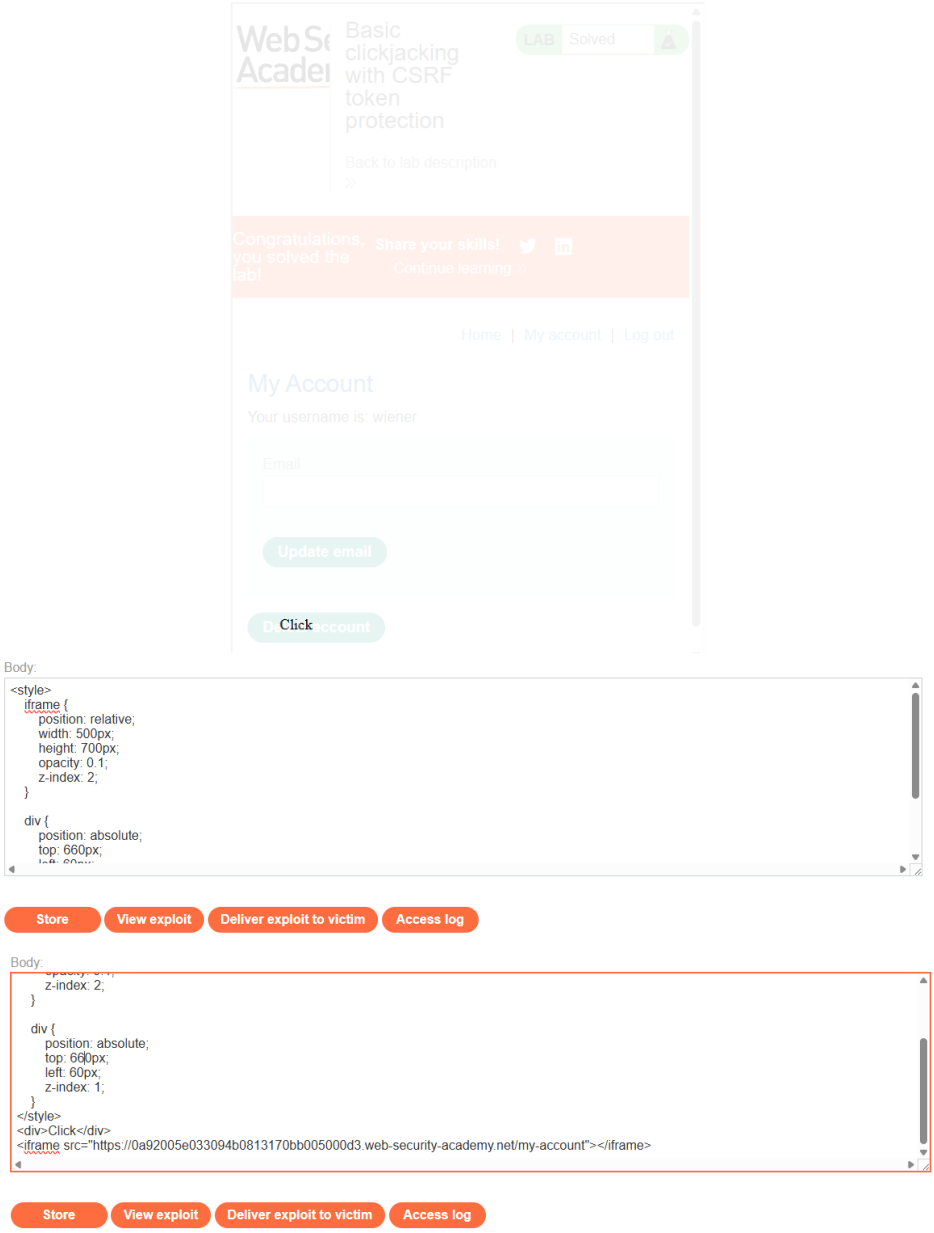
Borba protiv CSRF napada zahteva mehanizme koji razlikuju namerne akcije korisnika od onih koje iniciraju eksterni sajtovi:

1. **Anti-CSRF Tokeni (najsigurnija metoda):** Ovo je jedinstvena, kriptografski jaka i nepredvidljiva vrednost koju server generiše po sesiji ili po svakom zahtevu. Token se ubacuje u skriveno polje forme. Kada korisnik pošalje formu, server poredi primljeni token sa onim koji je sačuvan u sesiji. Napadač sa drugog sajta ne može pročitati ovaj token zbog *Same-Origin Policy* (SOP) pravila.
2. **SameSite Cookie atribut:** Postavljanje atributa na 'Lax' ili 'Strict' u `Set-Cookie` headeru. Ovo govori pregledaču da ne šalje kolačiće uz cross-site zahteve. 'Strict' potpuno zabranjuje slanje kolačića ako korisnik nije direktno na sajtu, dok 'Lax' dozvoljava samo bezbedne navigacije (poput GET linkova).
3. **Custom Request Headeri:** Korišćenje API-ja koji zahtevaju custom headere (npr. `X-Requested-With`). Pošto pregledači ne dozvoljavaju slanje custom headera preko domena bez CORS odobrenja, ovo je efikasna zaštita za moderne SPA aplikacije.
4. **Verifikacija korisnika za osetljive akcije:** Za operacije visokog rizika (npr. prenos novca), aplikacija treba da zahteva ponovni unos lozinke, rešavanje CAPTCHA koda ili 2FA validaciju, što je nemoguće simulirati putem CSRF-a.

6. Lab: Basic clickjacking with CSRF token protection

Poenta clickjacking-a je da napravimo "mamacštranicu ispred koje ćemo staviti transparentni iframe sajta čiju funkcionalnost želimo da izvršimo. Prednost je što će sada csrf token biti generisan jer se sajt učitava u sopstvenom sigurnosnom kontekstu (origin)

unutar iframe-a. Cilj zadatka je da preklopimo dugme iz iframe-a sa našim "mamac-đugmetom. Tako će se funkcionalnost brisanja korisnika izvršiti kada neko klikne na naš mamac.



Slika 7: Izgled "mamacštranice, bitna stavka da podesimo z index da je veći za iframe od elementa mamac stranice.